Thought for 11s

Yes — you can build agents with **OpenAI API, LangChain/LangGraph, CrewAI, plain Python, etc.**, but for the Fetch prize they need to be exposed as **uAgents / Agentverse agents** that speak the **Chat Protocol**.

Think of it like this:

```
ASI:One
  ↓ Chat Protocol
Agentverse-hosted uAgent wrapper
  ↓
Your actual logic:
  - OpenAI API
  - LangGraph
  - Google Calendar API
  - events.json
  - database
```

Agentverse is the **discovery + hosting/chat layer**. Your agent code can still call whatever APIs/frameworks you want.

Fetch's docs say the uAgents framework supports agents built with anything from LLMs to basic APIs, and Agentverse lets you create/host agents directly by choosing a blank or skeleton agent. uagents.fetch.ai Fetch.ai Innovati...

For your project, I'd do this:

Best setup for the hackathon

Option A — simplest and safest

Use **uAgents + plain Python**.

Each agent is basically:

```
receive message
parse intent
call tool/API
return structured response
```

Use:

- `uagents`
- OpenAI API or ASI model call
- Google Calendar API
- curated `events.json`
- simple Python functions

This is probably best for 36 hours.

Option B — stronger orchestration

Use **one uAgent wrapper + LangGraph inside it**.

```
ASI:One → Plans Orchestrator uAgent → LangGraph workflow → tools/ager
```

Fetch has examples for integrating LangGraph with uAgents, so this is a legitimate path.

Fetch.ai Innovati...

This gives you a cleaner multi-step flow:

1. identify group

2. get free times
3. rank events
4. draft proposal
5. book event

Option C — CrewAI

Possible, but I would not choose it unless your team already knows it. Fetch also has a CrewAI adapter example, but CrewAI can become slower/more abstract than you need.

Fetch.ai Innovati...

My recommendation

Build **one main Agentverse agent** first:

```
Plans Orchestrator Agent
```

It handles the full demo. Internally, make “agents” as Python modules:

```
calendar_agent.py
interest_agent.py
event_agent.py
proposal_agent.py
booking_agent.py
```

Then, only if time allows, split them into separately registered Agentverse agents.

Why? Because the prize requires agent(s), Chat Protocol, reasoning, tool execution, and real-world outcome. It does **not** require that every sub-agent be separately hosted. Your notes say the key deliverables are ASI:One chat URL, Agentverse agent URL, GitHub repo, and demo video. LA Hacks Preperation

Concrete stack

Use this:

```
Python
uAgents
Fetch Chat Protocol
OpenAI API
Google Calendar API
Pydantic
FastAPI optional
```

```
events.json
dotenv
```

Skip LangChain unless you specifically need it. For your use case, plain Python + OpenAI tool-style calls is enough.

Build order

1. Create a blank/skeleton Agentverse agent.
2. Implement Chat Protocol response: “Hello, I’m Plans.”
3. Add orchestrator logic.
4. Add fake calendar + fake event data.
5. Add OpenAI-generated proposal text.
6. Add real Google Calendar write.
7. Polish Agentverse profile keywords so ASI:One discovers it.

The main thing: **don’t start with LangChain. Start with a working uAgent on Agentverse.** Then plug OpenAI/Calendar/tools into it.

Sources

Sources

but i think langchain sounds more impressive/better on the resume. I've already used openai API and python and those tools are already listed on my resume, but not langchain. never done a proejct with langchian before.

Use **LangGraph**, not generic LangChain.

It gives you the resume keyword **and** it actually fits your project better.

Best resume-aligned choice

Build:

```
ASI:One
→ Agentverse / uAgents Chat Protocol wrapper
```

```
→ LangGraph multi-agent workflow
→ Google Calendar API + events.json + OpenAI API
```

On resume, this becomes:

> Built an Agentverse-discoverable multi-agent social coordination system using LangGraph, uAgents, OpenAI API, and Google Calendar API to autonomously revive dormant group chats by finding shared availability, matching events, and booking calendar plans.

That sounds much stronger than just “used OpenAI API.”

Why LangGraph over LangChain

LangChain is broad. LangGraph is the agent orchestration framework people associate with **stateful multi-agent workflows**.

Your project naturally has graph nodes:

```
Input / user intent
↓
Dormancy Detector
↓
Calendar Availability Node
↓
Interest Matching Node
↓
Event Ranking Node
↓
Proposal Writer Node
↓
Booking Executor Node
↓
Confirmation Node
```

That is literally a LangGraph workflow.

Recommended structure

```
plans-agent/
  agentverse/
    main.py              # uAgent + Chat Protocol wrapper
  graph/
    workflow.py          # LangGraph graph
    state.py             # shared state schema
    nodes/
```

```
      calendar_node.py
      interest_node.py
      event_node.py
      proposal_node.py
      booking_node.py
  data/
    events.json
    users.json
  services/
    google_calendar.py
    openai_client.py
```

How it should work

The **uAgent** is the public-facing agent on Agentverse.

The **LangGraph workflow** is the brain.

So when ASI:One sends:

> "Revive my LA Friends chat."

Your uAgent does:

```
result = plans_graph.invoke({
    "group_id": "la_friends",
    "intent": "revive dormant group chat"
})
```

Then LangGraph runs:

```
calendar_node → event_node → proposal_node → booking_node
```

and returns the final response to ASI:One.

Don't overdo it

Don't create 5 fully independent Agentverse agents unless you have extra time. That adds networking/debugging pain.

For hackathon + resume, this is enough:

```
1 Agentverse agent
+ internal LangGraph multi-agent orchestration
```

You can still honestly say “multi-agent workflow” because each LangGraph node can represent a specialist agent/tool role.

Final recommendation

Use **LangGraph + uAgents**.

Avoid full LangChain complexity unless necessary.

Your stack should be:

```
Python
uAgents
Fetch Chat Protocol
LangGraph
OpenAI API
Google Calendar API
Pydantic
events.json
```

This is both:

- better for the hackathon
- better for your resume
- not too risky to learn in 36 hours

so give me a step by step plan for how to build these agents out

Here’s the build plan I’d follow: **one Agentverse/uAgents wrapper + one internal LangGraph workflow**. That gets you Agentverse eligibility while keeping implementation manageable. Your own project brief already locks the MVP around 5 agents: Calendar, Interest, Event, Proposer, and Booking. LA Hacks Preperation

Phase 0 — Lock the exact hero flow

Build only this:

> ASI:One user asks: “Revive my LA Friends group chat.”
> Agent finds shared availability, picks a matching event, drafts a message, and books it on Google Calendar.

Do **not** build full chat, birthdays, auto-delete, sentiment ranking, or real event scraping yet. Your notes explicitly say the hero demo should be a dormant group revived into a real calendar event. LA Hacks Preperation

---

Phase 1 — Repo setup

Create this structure:

```
plans-agent/
  agentverse/
    main.py
  graph/
    workflow.py
    state.py
    nodes/
      calendar_node.py
      interest_node.py
      event_node.py
      proposal_node.py
      booking_node.py
  services/
    google_calendar.py
    openai_client.py
  data/
    users.json
    events.json
  README.md
  requirements.txt
  .env.example
```

Install:

```bash
pip install uagents langgraph langchain-openai pydantic python-dotenv goog
```

Phase 2 — Make the Agentverse agent work first

Before building intelligence, make sure ASI:One can talk to your agent.

`agentverse/main.py` should:

1. create a uAgent
2. implement Chat Protocol
3. respond with a hardcoded message

Goal:

```
ASI:One → your Agentverse agent → “Plans is online.”
```

Do this before LangGraph. Agentverse + Chat Protocol is a required prize qualifier.

LA Hacks Preperation

Phase 3 — Define shared LangGraph state

In `graph/state.py` , define the object every node updates:

Python Run

```python
from typing import TypedDict, List, Dict, Optional

class PlansState(TypedDict):
    user_query: str
    group_id: str
    group_members: List[str]
    free_windows: List[Dict]
    interests: Dict
    candidate_events: List[Dict]
    selected_event: Optional[Dict]
    proposal_message: Optional[str]
    booking_result: Optional[Dict]
    final_response: Optional[str]
```

This keeps every “agent” clean.

Phase 4 — Build nodes with fake data first

1. `interest_node.py`

Reads `data/users.json`.

```
Input: group_id
Output: interests
```

Example output:

```JSON
{
  "maya": ["art", "food"],
  "jordan": ["basketball", "food"],
  "priya": ["music", "coffee"],
  "andrew": ["food", "tech"]
}
```

2. `calendar_node.py`

Start with fake availability.

```
Output: Saturday 2–6pm shared free window
```

Do **not** start with OAuth. First make the whole graph run.

3. `event_node.py`

Reads `events.json`, filters by:

- time overlaps with free window
- location is LA/UCLA
- tags match group interests

4. `proposal_node.py`

Uses OpenAI to write the friendly message:

> "You four are all free Saturday 2–6pm. There's a gallery opening in Arts District at 3pm. Want me to book it?"

5. `booking_node.py`

Start fake:

```JSON
```

```
{
  "status": "booked",
  "message": "Added to all calendars."
}
```

Phase 5 — Wire the LangGraph workflow

In `graph/workflow.py` :

```
START
 → interest_node
 → calendar_node
 → event_node
 → proposal_node
 → booking_node
 → final_response_node
END
```

For the hackathon, keep it linear. Don't build complex routing unless you have time.

Your orchestration should look like:

Python Run

```python
from langgraph.graph import StateGraph, END
from graph.state import PlansState

def build_graph():
    graph = StateGraph(PlansState)

    graph.add_node("interests", interest_node)
    graph.add_node("calendar", calendar_node)
    graph.add_node("events", event_node)
    graph.add_node("proposal", proposal_node)
    graph.add_node("booking", booking_node)
    graph.add_node("final", final_node)

    graph.set_entry_point("interests")
    graph.add_edge("interests", "calendar")
    graph.add_edge("calendar", "events")
    graph.add_edge("events", "proposal")
    graph.add_edge("proposal", "booking")
    graph.add_edge("booking", "final")
    graph.add_edge("final", END)
```

```
    return graph.compile()
```

Phase 6 — Connect Agentverse → LangGraph

In your uAgent handler:

```python
result = plans_graph.invoke({
    "user_query": incoming_message,
    "group_id": "la_friends",
    "group_members": ["maya", "jordan", "priya", "andrew"],
})
```

Return:

```python
result["final_response"]
```

Now your ASI:One demo actually triggers the multi-agent workflow.

Phase 7 — Replace fake calendar with Google Calendar

Only after fake end-to-end works.

Implement:

```
services/google_calendar.py
```

Functions:

```python
get_free_busy(user_emails, time_min, time_max)
create_calendar_event(user_emails, event)
```

For the demo:

- pre-auth team accounts
- use one known weekend window

- have fallback fake mode if Google API breaks

Your notes correctly flag Google Calendar write as the riskiest demo moment.

LA Hacks Preperation

Phase 8 — Make the agents feel real in ASI:One

Have the final response show the execution trace:

```
I found a shared window: Saturday 2–6pm.
I matched it to: Arts District Gallery Opening at 3pm.
I drafted the group message.
I added the event to all 4 calendars.

Message to send:
“Hey — this chat has been quiet, but you’re all free Saturday...”
```

This makes the multi-agent orchestration obvious to judges.

Phase 9 — Polish Agentverse profile

Use keywords like:

```
group scheduling
friendship
calendar coordination
event planning
social agent
dormant group chat
```

Your agent should be discoverable from prompts like:

> “Find an agent that helps me coordinate plans with friends.”

Phase 10 — Demo safety

Prepare two modes:

```
LIVE_MODE=true
```

Uses Google Calendar API.

```
LIVE_MODE=false
```

Uses fake calendar results but identical output.

Record the perfect live run before submission.

Build order summary

1. Stub uAgent on Agentverse
2. Hardcoded ASI:One response
3. LangGraph state + fake nodes
4. End-to-end fake workflow
5. OpenAI proposal generation
6. Google Calendar free/busy
7. Google Calendar event creation
8. Final ASI:One demo
9. README + demo video

That's the fastest path to a project that is both **prize-eligible** and resume-worthy.

Sources

Sources